# Semaphores

↳ oldest and more general synchronization primitive

(invented first by Dijkstra in 1960s)

Locks and condition variables came later as more specialized, safer and clearer abstractions.

**Goal**: How to use semaphores?

A semaphore is an object with an integer value that we can manipulate with two routines

- sem_wait()
- sem_post()

a crucial part while using semaphores → its initialization.

```
1  #include <semaphore.h>
2  sem_t s;
3  sem_init(&s, 0, 1);
```

imports POSIX semaphore library functions like sem_init(), sem_wait(), sem_post() etc...

declares a semaphore variable s of type sem_t (it is like a small struct that internally keeps a counter (integer value) and a queue of waiting threads)

initialization of the semaphore s.

0 → means it is a thread-shared semaphore (if its 1, it could be used across process with shared memory)

1 → initial value of the semaphore counter.

# sem_wait() and sem_post() routines

```
1  int sem_wait(sem_t *s) {
2      decrement the value of semaphore s by one
3      wait if value of semaphore s is negative
4  }
5
6  int sem_post(sem_t *s) {
7      increment the value of semaphore s by one
8      if there are one or more threads waiting, wake one
9  }
```

→ return right away (value at s is >0 when we called)

→ cause the caller to suspend execution and wait.

Remark: Note that the value of the semaphore when negative, is equal to the # waiting threads.

## 1. Binary semaphores (using semaphore as a lock)

```
1  sem_t m;
2  sem_init(&m, 0, X); // init to X; what should X be?
3
4  sem_wait(&m);
5  // critical section here
6  sem_post(&m);
```

To use the above semaphore as a lock, the value of X should be 1.

Why? consider the following scenario;
let T0 and T1 be two threads. → value of semaphore m

- first T0 calls sem_wait() → since m.value = 1 currently,
it decrements it by 1 ⇒ m.value = 0 ≥ 0
⇒ sem_wait() returns and thread continue.
⇒ free to enter the critical section.

- T0 calls sem_post() ⇒ m.value = 1 → sem_post() returns.

| Value of Semaphore | Thread 0 | Thread 1 |
|---|---|---|
| 1 | | |
| 1 | call `sem_wait()` | |
| 0 | `sem_wait()` returns | |
| 0 | `(crit sect)` | |
| 0 | call `sem_post()` | |
| 1 | `sem_post()` returns | |

what happens if T1 calls sem_wait() before T0 calls sem_post()?

T1 calls sem_wait() ⇒ decrement the current value (0)
⇒ m.value = -1 < 0 ⇒ T1 waits (go to sleep)

later T0 runs again → calls sem_post() → increments the value of semaphore

$\Rightarrow$ m.value = 0 and wakes the waiting thread (T1).
$\rightarrow$ T1 now enters the critical section

When T1 finishes $\rightarrow$ call sem_post() $\rightarrow$ m.value = 1 again!

| Val | Thread 0 | State | Thread 1 | State |
|---|---|---|---|---|
| 1 | | Run | | Ready |
| 1 | call sem_wait() | Run | | Ready |
| 0 | sem_wait() returns | Run | | Ready |
| 0 | (crit sect begin) | Run | | Ready |
| 0 | *Interrupt; Switch→T1* | Ready | | Run |
| 0 | | Ready | call sem_wait() | Run |
| -1 | | Ready | decr sem | Run |
| -1 | | Ready | (sem<0)→sleep | Sleep |
| -1 | | Run | *Switch→T0* | Sleep |
| -1 | (crit sect end) | Run | | Sleep |
| -1 | call sem_post() | Run | | Sleep |
| 0 | incr sem | Run | | Sleep |
| 0 | wake(T1) | Run | | Ready |
| 0 | sem_post() returns | Run | | Ready |
| 0 | *Interrupt; Switch→T1* | Ready | | Run |
| 0 | | Ready | sem_wait() returns | Run |
| 0 | | Ready | (crit sect) | Run |
| 0 | | Ready | call sem_post() | Run |
| 1 | | Ready | sem_post() returns | Run |

Why the above semaphore behaves as a lock? $\rightarrow$ a thread goes to sleeping state whenever another thread is inside the critical section (wrapped around by sem_wait() and sem_post())

Why the above is called a binary semaphore? $\rightarrow$ Because it is used as a lock here, which has only 2 states $\rightarrow$ held and not held.

## 2. Semaphores for ordering

Recall the first problem using condition variables
→ a parent waiting for a child to finish.
needs to wait do Signaling

How to achieve this using a semaphore?
parent calls sem_wait() child calls sem_post()

```
1  sem_t s;
2
3  void *child(void *arg) {
4      printf("child\n");
5      sem_post(&s); // signal here: child is done
6      return NULL;
7  }
8
9  int main(int argc, char *argv[]) {
10     sem_init(&s, 0, X); // what should X be?
11     printf("parent: begin\n");
12     pthread_t c;
13     Pthread_create(&c, NULL, child, NULL);
14     sem_wait(&s); // wait here for child
15     printf("parent: end\n");
16     return 0;
17 }
```

→ the value of X should be 0
Why?

case-1: parent creates the child → continues execution
(child has not run yet!)
parent calls sem_wait()
→ s.value = -1 ⇒ go to sleep
possible only if X=0 initially.
→ child runs, calls sem_post()
→ s.value = 0, wakes up parent → parent runs
→ finish the program.

Case-2: parent creates the child and the child immediately starts running.

child calls sem_post() → increment the value by 1
⇒ s.value = 1

when parent gets to run → parent calls sem_wait
→ decrement the value by 1
⇒ s.value = 0 ≥ 0

⇒ return from sem_wait() without waiting, as desired in this case

## 3. The Producer/consumer (Bounded buffer) problem

Recall the problem from the previous lecture.

First attempt:

The code for put() and get() routines.

```
1  int buffer[MAX];
2  int fill = 0;
3  int use  = 0;
4
5  void put(int value) {
6      buffer[fill] = value;     // Line F1
7      fill = (fill + 1) % MAX; // Line F2
8  }
9
10 int get() {
11     int tmp = buffer[use];    // Line G1
12     use = (use + 1) % MAX;    // Line G2
13     return tmp;
14 }
```

```
1  sem_t empty;
2  sem_t full;
3
4  void *producer(void *arg) {
5      int i;
6      for (i = 0; i < loops; i++) {
7          sem_wait(&empty);          // Line P1
8          put(i);                    // Line P2
9          sem_post(&full);           // Line P3
10     }
11 }
12
13 void *consumer(void *arg) {
14     int tmp = 0;
15     while (tmp != -1) {
16         sem_wait(&full);           // Line C1
17         tmp = get();               // Line C2
18         sem_post(&empty);          // Line C3
19         printf("%d\n", tmp);
20     }
21 }
22
23 int main(int argc, char *argv[]) {
24     // ...
25     sem_init(&empty, 0, MAX); // MAX are empty
26     sem_init(&full, 0, 0);    // 0 are full
27     // ...
28 }
```

sem_t empty; → tracks the empty slots in the buffer

sem_t full; → tracks the filled slots in the buffer

sem_wait(&empty); // Line P1 → wait until there is an empty slot (decrements empty count, so that if empty==0, producer blocks)

put(i); // Line P2 → produce an item to the buffer

sem_post(&full); // Line P3 → increment the count of full slots (if any consumer was waiting for items, one wakes up)

sem_wait(&full); // Line C1 → decrements full count, so that if full==0, consumer blocks

tmp = get(); // Line C2 → removes an item from buffer

sem_post(&empty); // Line C3 → increments empty count (if producer waiting → one wakes up)

sem_init(&empty, 0, MAX); // MAX are empty → buffer starts empty → MAX empty slots

sem_init(&full, 0, 0); // 0 are full → buffer starts with 0 full slots

Exercise: Try different cases with MAX = 1, it works properly (even with multiple producers & consumers)

What possibly happen when MAX > 1 ? (with multiple producers and consumers)

↳ a race condition

Because, the empty and full semaphores only tracks how many slots are empty and full, not who is accessing the buffer.
i.e., they control how many produces or consumers can proceed, but not mutual exclusion on the buffer data structure itself.

For instance, let buffer size = 10 ($\Rightarrow$ MAX = 10)
at some point, let empty = 8 and full = 2.

8 free slots  2 empty slots.

Now, let's say two producers $P_A$ and $P_B$ calls sem_wait(&empty)

**Step-by-step interleaving**

| Step | Action | empty | full | Notes |
|---|---|---|---|---|
| 1 | Pa calls `sem_wait(&empty)` | 7 | 2 | Pa enters `put()` |
| 2 | Pa starts `put()` — writes at index `fill = 0` | | | Not yet incremented |
| 3 | Pa **gets interrupted** mid-way | | | |
| 4 | Pb calls `sem_wait(&empty)` | 6 | 2 | Allowed — because there are still empty slots left! |
| 5 | Pb enters `put()` — also writes at index `fill = 0` | | | Overwrites Pa's item |

```
1  int buffer[MAX];
2  int fill = 0;
3  int use  = 0;
4
5  void put(int value) {
6      buffer[fill] = value;     // Line F1
7      fill = (fill + 1) % MAX; // Line F2
8  }
9
10 int get() {
11     int tmp = buffer[use];    // Line G1
12     use = (use + 1) % MAX;    // Line G2
13     return tmp;
14 }
```

Why did this happen? sem_wait(&empty) only ensures that there exists an empty slot, not that "only one producer" is modifying the buffer's internal data structures.

In fact, there is no mutual exclusion on
- the fill index variable inside put()
- the actual array buffer[]

so both producers can be inside put() simultaneously.

what can we do to fix this problem? → use binary semaphores to add locks?

```
1  void *producer(void *arg) {
2      int i;
3      for (i = 0; i < loops; i++) {
4          sem_wait(&mutex);       // Line P0 (NEW LINE)
5          sem_wait(&empty);       // Line P1
6          put(i);                 // Line P2
7          sem_post(&full);        // Line P3
8          sem_post(&mutex);       // Line P4 (NEW LINE)
9      }
10 }
11
12 void *consumer(void *arg) {
13     int i;
14     for (i = 0; i < loops; i++) {
15         sem_wait(&mutex);       // Line C0 (NEW LINE)
16         sem_wait(&full);        // Line C1
17         int tmp = get();        // Line C2
18         sem_post(&empty);       // Line C3
19         sem_post(&mutex);       // Line C4 (NEW LINE)
20         printf("%d\n", tmp);
21     }
22 }
```

Does this work?

↳ unfortunately, NO!

why? a deadlock!

How?

Consider the following scenario.

Imagine two threads, one producer and one consumer.

Suppose the consumer gets to run first
→ it acquires mutex
→ calls sem_wait() on the full semaphore
→ decrement value ⇒ full.value = -1 < 0
(initial value of full was 0)
⇒ consumer gets blocked.
problem: consumer still holds the lock!

Now, suppose producer runs
→ calls sem_wait on mutex
→ gets stuck!
→ a classic deadlock!

(instead, here if it could wake up the consumer thread, all would have been good).

(as the consumer holds the mutex and is waiting for someone to signal full. The producer could signal full, but is waiting for the mutex. ie., each of them are stuck waiting for each other!)

How to fix this? → reduce the scope of the lock!

```
1  void *producer(void *arg) {
2      int i;
3      for (i = 0; i < loops; i++) {
4          sem_wait(&empty);          // Line P1
5          sem_wait(&mutex);          // Line P1.5 (lock)
6          put(i);                    // Line P2
7          sem_post(&mutex);          // Line P2.5 (unlock)
8          sem_post(&full);           // Line P3
9      }
10 }
11
12 void *consumer(void *arg) {
13     int i;
14     for (i = 0; i < loops; i++) {
15         sem_wait(&full);           // Line C1
16         sem_wait(&mutex);          // Line C1.5 (lock)
17         int tmp = get();           // Line C2
18         sem_post(&mutex);          // Line C2.5 (unlock)
19         sem_post(&empty);          // Line C3
20         printf("%d\n", tmp);
21     }
22 }
```

Finally, a working solution for the bounded buffer problem.